

Создайте граф знаний на основе текста с помощью Python

Узнайте, как создать граф знаний на Python с помощью CogDB. Извлекайте сущности и связи из текста с помощью большой языковой модели, загружайте их в виде троек и выполняйте запросы с помощью обхода графа и векторного поиска.

Что такое граф знаний?

Граф знаний — это структура данных, представляющая информацию в виде сети. Объекты моделируются в виде узлов, а связи между ними — в виде ребер.

В этом руководстве мы возьмем обычный текстовый документ о пригодности планет для жизни, с помощью большой языковой модели извлечем сущности и связи между ними, загрузим их в CogDB и создадим граф знаний.

После того как граф будет построен, мы выполним на нем запросы и зададим несколько интересных вопросов.

[Исходный код на GitHub](#)

Исходный материал

В качестве исходного материала мы будем использовать статью в Википедии *Обитаемость планет Солнечной системы*:

https://en.wikipedia.org/wiki/Planetary_habitability_in_the_Solar_System

Исходный текст, использованный в этом руководстве, доступен в `planetary-habitability.txt` внутри репозитория GitHub.

Определение онтологии

Чтобы создать граф знаний, сначала нужно определить онтологию. Онтология определяет типы сущностей и связей, допустимых в графе. Ее можно рассматривать как схему графа, подобно тому, как схема базы данных определяет структуру реляционной базы данных.

Онтология придает смысл узлам и связям.

Данные будут храниться в виде триплетов «субъект — предикат — объект» (SPO):

- Подлежащее → узел
- Сказуемое → ребро
- Дополнение → узел

Узлы представляют собой сущности, а рёбра — связи между ними.

Поскольку мы работаем в области планетологии и космических исследований, мы начнем с определения некоторых типов сущностей. Для таких хорошо изученных областей, как наша, большие языковые модели могут быть весьма полезны для создания базовой онтологии.

Если мы дадим большой языковой модели исходный текст и попросим ее определить категории сущностей, то можем получить что-то вроде этого:

```
- CelestialBody
- Moon
- Mission
- Spacecraft
- SpaceAgency
- Scientist
- Chemical
- Feature
- Concept
- Region
- Instrument
```

- Event

Далее мы определяем связи, которые могут объединять эти сущности:

- ORBITS
- MOON_OF
- HAS_FEATURE
- HAS_ATMOSPHERE
- EXPLORED_BY
- OPERATED_BY
- LAUNCHED_IN
- CARRIES
- DISCOVERED
- INDICATES
- REQUIRES
- LOCATED_IN
- MAY_HARBOR
- CONTAINS
- SUCCESSOR_OF
- TARGETS
- EVIDENCE_FOR
- PROPOSED_BY
- PART_OF

Extracting Entities and Relationships

With the ontology defined, we can extract structured data from the raw text.

LLMs work well for this kind of task. We provide the ontology and ask the model to extract entities that conform to it.

Here's an example:

```
def extract_entities(client, text: str) -> list[dict]:
    """Extract typed entities from text, guided by the ontology."""
    response = client.chat.completions.create(
        model="gpt-5",
        messages=[
            {
                "role": "system",
                "content": (
                    "You are a knowledge graph entity extractor.\n\n"
                    "Given a text about planetary science and astrobiology, extract "\n\n"
                    "all named entities. For each entity return:\n\n"
                    '  {"name": "<short canonical name>", "type": "<EntityType>"}\n\n'
                    "Use these entity types ONLY:\n"
                    + "\n".join(f"- {k}: {v}" for k, v in ONTOLOGY["entity_types"].items())
                )
            },
            {
                "role": "user",
                "content": text
            }
        ]
    )
    return response.choices[0].message.content
```

```

        + "\n\nRules:\n"
        "- Use short canonical names: 'Europa' not 'Europa, a moon of Jupi
        "- Normalize: 'JWST' and 'James Webb Space Telescope' → 'JWST'\n"
        "- One entry per unique entity\n"
        "- Return ONLY a JSON array. No commentary.\n"
    ),
    },
    {"role": "user", "content": text},
],
)
raw = response.choices[0].message.content.strip()
if raw.startswith("```"):
    raw = raw.split("\n", 1)[1].rsplit("```", 1)[0]
return json.loads(raw)

```

Normalization

After extraction, we typically need to normalize entities and relationships. Different phrases may refer to the same concept, and we want them represented consistently in the graph.

For example:

```

- james webb space telescope -> jwst
- james webb -> jwst
- webb telescope -> jwst
- webb -> jwst
- jst -> jwst
- perseverance rover -> perseverance
- perseverance -> perseverance
- cassini-huygens -> cassini
- cassini spacecraft -> cassini

```

Normalization can be handled either by prompting the LLM carefully or by post-processing its output.

Loading Triples into CogDB

Once we have triples, we can load them into CogDB.

Here is a simplified example:

```
from cog.torque import Graph

triples = [
    ("Europa", "ORBITS", "Jupiter"),
    ("Europa", "HAS_FEATURE", "Subsurface_Ocean"),
    ("Europa", "MAY_HARBOR", "Life"),
]

g = Graph("AstrobiologyKG")

g.put_batch(triples)
```

We can also attach vector embeddings to entities to enable semantic search. In the demo code, embeddings are generated using OpenAI's embeddings API.

```
# dummy embeddings for demonstration purposes.
g.put_embedding("europa", [0.1, 0.2, 0.3])
g.put_embedding("jupiter", [0.1, 0.2, 0.3])
g.put_embedding("subsurface ocean", [0.1, 0.2, 0.3])
g.put_embedding("life", [0.1, 0.2, 0.3])
```

Querying a Knowledge Graph in Python

Now we have a working knowledge graph. We can traverse it, run semantic searches, or combine both.

Below are a few example queries. [source code on GitHub](#)

► Try these queries in the Playground [↗](#)

A. Graph Traversal

What is Europa connected to in the graph?

```
g.v("europa").out().all()
```

life, subsurface ocean

Which entities mention or relate to water?

```
g.v("water").inc().all()
```

messenger

Where is Jezero Crater located, and what might that body harbor?

```
g.v("jezero crater").out("located_in").out("may_harbor").all()
```

life

Which missions target bodies that may harbor life?

```
g.v("life").inc("may_harbor").inc("targets").all()
```

europa clipper, dragonfly

B. Semantic Search

Nearest entities to "signs of life"

```
g.v().k_nearest("signs of life", k=5).all()
```

biosignature, dawn, massapanspermia, mars, panspermia

Nearest entities to "ocean world"

```
g.v().k_nearest("ocean world", k=5).all()
```

subsurface ocean, europa, caloris basin, moon, mars

► Попробуйте выполнить эти запросы на игровой площадке [↗](#)

С. Гибридный подход: обход графа + семантическая фильтрация

Какие объекты в радиусе двух переходов от Европы наиболее тесно связаны с «водой и льдом»?

```
g.v("europa").dfs(max_depth=2).k_nearest("water and ice", k=3).all()
```

subsurface ocean

Which bodies may harbor life, ranked by relevance to "extraterrestrial biology"?

```
g.v("life").inc("may_harbor").k_nearest("extraterrestrial biology", k=5).all()
```

mars, titan, europa

What is Enceladus connected to that is most related to "ocean"?

```
g.v("enceladus").out().k_nearest("ocean", k=3).all()
```

subsurface ocean

What do NASA spacecraft target, ranked by "habitable world"?

```
g.v("nasa").inc("explored_by").out("targets").k_nearest("habitable world", k=5).all()
```

europa

Find entities similar to "icy moon", then see what they may harbor

```
g.v().k_nearest("icy moon", k=3).out("may_harbor").all()
```

early life, life, subsurface ocean

Which missions target habitable bodies, ranked by "robotic spacecraft"?

```
g.v("life").inc("may_harbor").inc("targets").k_nearest("robotic spacecraft", k=3).all()
```

dragonfly, europa clipper

Find "space exploration" entities, follow their edges, then filter for "habitable world"

```
g.v().k_nearest("space exploration", k=10).out().k_nearest("habitable world", k=5).all()
```

subsurface ocean, curiosity, gale crater, europa, titan

Find "deep space probe" entities, see what they target, then filter for "frozen world"

```
g.v().k_nearest("deep space probe", k=5).out("targets").k_nearest("frozen world", k=3)
```

gale crater

► Попробуйте все запросы на игровой площадке [↗](#)

< Guides

Previous Page

Semantic Search Over a Movie Knowledge
Graph >

Combine graph traversals with semantic se...